

ESCAPARATES PRO

ENTERPRISE MASTER SPECIFICATION

Producto · arquitectura · seguridad · billing · IA · legal · roadmap

Documento normativo para implementación verificable con Codex y Claude

Versión 2.0 — Audited Edition

Baseline GitHub: master @ cc3a8002 · 14 julio 2026

Nivel objetivo: paid beta segura → SaaS comercial → enterprise readiness

Preparado para Juan Manuel Espinosa Galant

CONFIDENCIAL · WORKING SPEC



Control documental

Documento
Escaparates Pro — Enterprise Master Specification

Versión
2.0 — Audited Edition

Fecha
14 de julio de 2026

Propietario de producto
Juan Manuel Espinosa Galant

Repositorio auditado
Juanmaes83/escaparates-pro

Rama / commit base
master / cc3a8002d0a5ab5237ca87068990d03f0081e54c

Estado
Normativo para planificación; implementación sujeta a PR y verificación

Audiencia
Producto, ingeniería, seguridad, legal, operaciones, Codex, Claude y revisores

Clasificación
Confidencial — working specification

Propósito. Convertir Escaparates Pro en un SaaS comercial real, seguro, medible y escalable sin destruir el activo diferencial: el editor y runtime visual existente. Este documento sustituye al roadmap lineal de la versión 1.0 como guía operativa y lo transforma en una especificación ejecutable por agentes de ingeniería.

Autoridad y jerarquía de fuentes

Esta especificación define objetivos, invariantes, arquitectura y release gates.

Los contratos ejecutables —OpenAPI, JSON Schema, migraciones y tests— son la verdad técnica de cada versión.

docs/status/current-state.md debe reflejar el commit desplegado y nunca aspiraciones futuras.

Los ADR documentan decisiones irreversibles o costosas.

El README explica uso; no sustituye contratos ni estado.

Regla anti-deriva: una PR que cambie comportamiento, schema, plan, endpoint, seguridad o despliegue debe actualizar la documentación correspondiente en la misma PR.

1. Resumen ejecutivo

1.1 Veredicto profesional

Escaparates Pro ya no es sólo un prototipo visual. El repositorio auditado contiene una API Fastify/TypeScript, PostgreSQL con Drizzle, autenticación email/password, sesiones persistidas, workspace gratuito por defecto, una base de Stripe Checkout y un frontend que exige login y lee el plan desde backend [G1–G7]. El PR #11 fue fusionado el 14 de julio de 2026 y confirma que el flujo visual ya distingue Demo, Free y Pro [G2].

Sin embargo, **todavía no es un SaaS preparado para cobrar dinero real de forma segura y repetible**. La brecha principal no está en añadir más efectos; está en cerrar cinco sistemas de verdad:

Identidad segura: el refresh token sigue almacenado en localStorage, por tanto es extraíble ante XSS [G4].

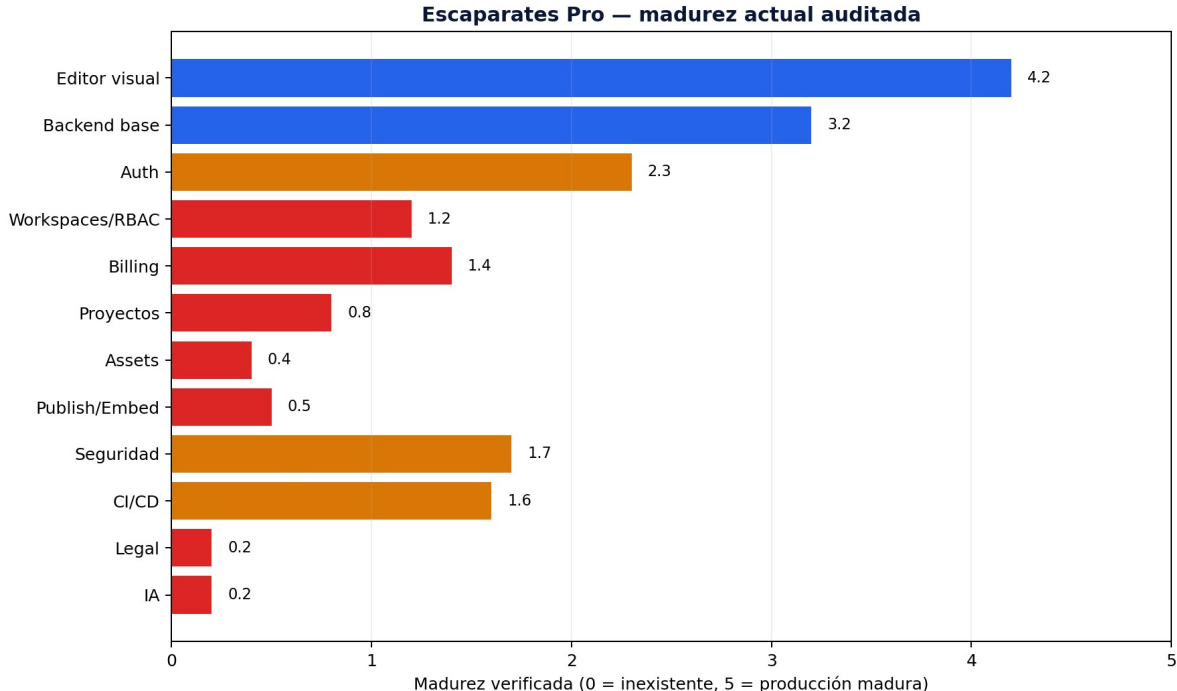
Billing autoritativo: existe Checkout, pero no webhook, portal, reconciliación ni lifecycle de suscripción [G6].

Entitlements server-side: el frontend bloquea acciones, pero el backend todavía no protege cada recurso y cuota mediante un resolver canónico [G5].

Persistencia y delivery: proyectos, presets, assets, publicaciones y exports no forman aún un flujo cloud completo [G3, G7].

Trust readiness: faltan legal center, licencias/provenance, aceptación versionada, observabilidad, backups verificados y quality gates completos.

Conclusión: el producto está en una fase de *pre-paid-beta*. La calidad visual es un activo de primer nivel; la madurez SaaS global es aproximadamente **2,0/5**. El siguiente objetivo no es “enterprise features”, sino **enterprise-grade execution discipline** para lanzar una paid beta segura.



Madurez auditada

1.2 Decisión estratégica central

Escaparates Pro debe posicionarse como:

Plataforma de producción y entrega de experiencias visuales comerciales, que convierte imágenes y vídeos de una marca en piezas interactivas, publicables y reutilizables sin depender de WebGL, motion design o desarrollo a medida.

No debe venderse como una colección de efectos. El usuario compra un resultado: un hero, campaña, módulo web, embed, pieza de signage o asset social listo para usar.

1.3 Cuña inicial de mercado

La cuña recomendada es más estrecha que la versión 1.0:

agencias web pequeñas;

freelancers y diseñadores web;

estudios creativos con varios clientes;

ecommerce y marcas visuales con campañas recurrentes.

El *job to be done* inicial es:

“Con los assets que ya tengo, necesito crear, aprobar y publicar una experiencia visual premium para un cliente o campaña en menos de 20 minutos, sin programar y sin entregar el editor.”

Retail, hoteles, inmobiliarias y restaurantes son verticales de demostración, no productos separados durante el MVP.

1.4 North Star y promesa operativa

North Star: experiencias profesionales publicadas o exportadas con éxito por workspace activo al mes.

Una experiencia cuenta sólo si:

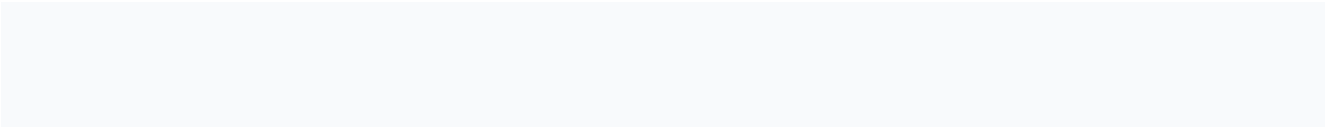
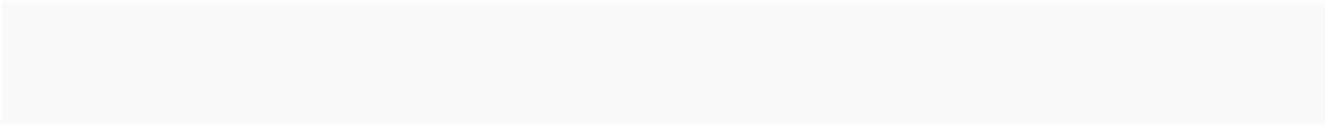
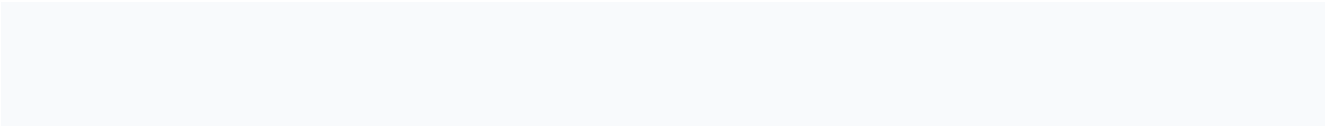
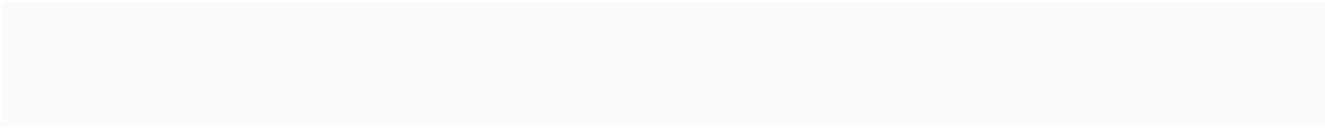
nace de un proyecto persistido;

supera validación;

genera un export descargable o una publicación estable;

no contiene el editor;

está vinculada a un workspace y a un evento de uso auditable.



2. Qué cambia respecto al Documento Maestro v1.0

La versión 1.0 definía correctamente el objetivo de convertir el editor en SaaS, proteger el frontend, añadir backend, storage, pagos y automatización [B1]. Su principal limitación es temporal: describía backend, base de datos, auth y pagos como inexistentes, mientras el repositorio ya ha implementado parcialmente esas fases. También mezclaba visión, hipótesis comerciales y tareas técnicas sin contratos de aceptación.

Estado actual
Snapshot anterior al backend

Auditoría por commit, PR y archivo

Roadmap

15 fases lineales

Release gates, dependencias, Definition of Done y triggers

Arquitectura

Componentes generales

Monolito modular, contratos, estados y límites

Datos

Lista conceptual de tablas

Modelo operativo, ledger, outbox, legal y AI runs

Billing

Stripe + webhooks como objetivo

Lifecycle completo, reconciliación y entitlement projection

Seguridad

Checklist general

P0 concretos sobre tokens, URLs, uploads, tenant y supply chain

IA

Descripción funcional

Pipeline estructurado, evaluación, coste, provenance y no-code execution

Legal

GDPR genérico

Legal & Trust Center, aceptación versionada, DPA, cookies, IP y takedown

Agentes

“Codex implementa, Claude revisa”

Contrato de tarea, ramas, PR, evidencias, no-go y revisión independiente

Pricing

Cinco planes desde el inicio

Catálogo objetivo separado del catálogo de lanzamiento

Proyecciones

Escenarios MRR ilustrativos

Modelo de inputs, unit economics y experimentos falsables

3. Estado real del repositorio

3.1 Baseline auditada

Repositorio público: Juanmaes83/escaparates-pro.

Rama por defecto: master.

Commit auditado: cc3a8002..., merge del PR #11.

Frontend: aplicación visual estática y modular basada en scripts globales window.EP, con múltiples modos y viewers en iframe.

Backend: apps/api, Node 20, TypeScript, Fastify, Drizzle, PostgreSQL, Zod, Pino y Stripe.

Despliegue observado: previews Vercel para frontend y API staging en Railway.



3.2 Matriz “implementado /
parcial / no implementado”

Implementado avanzado

README enumera efectos, 25 módulos web y blueprints [G3]

Diferenciador principal; proteger

API foundation

Implementado

Helmet, CORS allowlist, rate limit, request ID y rutas [G8]

Buena base, aún sin modularización de dominio

PostgreSQL y migraciones

Implementado base

Tablas SaaS iniciales y migraciones versionadas [G7]

Faltan tablas operativas críticas

Auth email/password

Parcial

Register/login/me/logout y token hasheado en DB [G10]

Falta verificación, reset, rotación, cookie segura y sesiones por dispositivo

Workspace por defecto

Parcial

Se crea workspace Free y miembro owner [G11]

La lectura primaria sólo busca workspaces propiedad del usuario

Plan gate frontend

Parcial

Demo/Free/Pro y bloqueo visual [G5]

No es control de seguridad ni cuota autoritativa

Stripe Checkout

Parcial

Un único plan pro y una price env [G6]

Sin webhooks, portal, impuestos ni reconciliación

Proyectos

Schema parcial

projects y project_versions existen [G7]

Sin endpoints, autosave, conflicto, importación ni UI cloud

Presets

Schema parcial

Tabla existe; frontend conserva ep_presets_v1

Sin sync ni migración segura

Assets

No SaaS

Media vive en data URLs/object URLs del navegador

Sin storage, seguridad, cuotas o lifecycle

Publish / embed

Demo local

README indica URL local temporal [G3]

Sin release inmutable, revocación o CDN

Export registry/worker

No implementado

Export browser-only

Sin historial, jobs, retry, coste o artifacts

Créditos

No implementado

Sólo concepto y usage counters genéricos

Requiere ledger append-only

Observabilidad

Base mínima

Logs estructurados y request ID

Sin trazas, métricas, alertas ni SLOs

CI/CD

Parcial

CI ejecuta install, typecheck y build [G9]

No ejecuta tests, lint, seguridad ni E2E

Legal/compliance

No implementado

No se encontró legal center ni registros de aceptación

Bloqueador para pagos públicos

IA Campaign Builder

No implementado

Visión de producto

Debe esperar a contratos y coste



3.3 Hallazgos P0

- AUTH-P0-01
 - Refresh token en localStorage [G4]
 - Robo de sesión vía XSS
 - Cookie Secure HttpOnly same-site o access token corto en memoria + refresh cookie rotatorio
- BILL-P0-01
 - Checkout sin webhook [G6]
 - Acceso desincronizado, fraude, impagos no reflejados
 - Endpoint firmado, event store, handlers idempotentes, reconciliación
- BILL-P0-02
 - successUrl y cancelUrl aportadas por cliente [G6]
 - Open redirect / abuso de confianza
 - Allowlist de orígenes y construcción server-side
- ENT-P0-01
 - Plan y límites principalmente en navegador [G5]
 - Bypass trivial
 - Resolver server-side por workspace y middleware por feature/quota
- TENANT-P0-01

Workspace primario sólo por owner [G11]

Equipos y scoping incorrectos

Contexto de workspace seleccionado + membresía + rol

CI-P0-01

CI no ejecuta npm test [G9]

Merge de regresiones conocidas

Tests requeridos + integración PostgreSQL + smoke frontend

DOC-P0-01

Documentos de arquitectura desactualizados

Agentes implementan sobre premisas falsas

current-state.md, ADRs y OpenAPI en la misma PR

LEGAL-P0-01

Ausencia de Legal & Trust Center

Cobro y tratamiento de datos sin base contractual clara

Documentos, CMP, aceptación versionada y revisión jurídica

LIC-P0-01

Repositorio público y estrategia de licencia ambigua

Copia, contribución y notices inconsistentes

Decidir private/proprietary/open-core y registrar terceros

OPS-P0-01

Endpoint HTTP de migración en staging y operación manual

Superficie administrativa y drift

Job/CLI controlado, locks, checksums y bloqueo total en producción

4. Estrategia de producto

4.1 Taxonomía del producto

La amplitud actual debe convertirse en una arquitectura de producto comprensible:

Studio: editor y runtime visual.

Catalog: efectos, scroll sections, website modules, blueprints y data stories.

Projects: persistencia, versiones, presets, carpetas y clientes.

Assets: media privada, variantes, reutilización y derechos.

Publish: URL estable, release, embed, revocación y analytics.

Export: formatos descargables, historial y jobs premium.

Review: enlaces de cliente, comentarios y aprobación.

AI Director: brief → CampaignPlan → variantes de ProjectDocument.

Admin & Trust: billing, equipo, auditoría, legal, soporte y estado.

Esta taxonomía debe reflejarse en navegación, dominios backend, entitlements, métricas y documentación.

4.2 Posicionamiento

Mensaje principal

Convierte los assets de una marca en experiencias visuales premium listas para publicar, embeber o entregar a clientes.

Mensaje para agencias

Crea, aprueba y reutiliza visuales interactivos para clientes sin depender de motion designers o desarrolladores WebGL.

Lo que no debe prometer

“Crea cualquier web completa con IA”.

“Sustituye Canva, Webflow o After Effects”.

“4K ilimitado” sin coste medido.

“Enterprise” antes de disponer de seguridad, soporte y procesos.

Resultados de conversión o ventas no demostrados.

4.3 Flujo de activación

Registro y verificación.

Selección de objetivo: hero, campaña, embed, signage o social.

Template/effect recomendado.

Assets demo o upload propio.

Personalización mínima.

Preview válida.

Guardado automático del proyecto.

Export preview o publicación de prueba.

Upsell contextual a Pro.

Evento de activación: project_first_value_completed, emitido cuando el usuario guarda un proyecto y consigue un export/publicación válida dentro de las primeras 24 horas.

4.4 Principios de producto

Time-to-value menor de 10 minutos para una demo y menor de 20 para un asset propio.

Progressive disclosure: controles avanzados sólo cuando aportan valor.

Default de calidad: la plantilla debe verse bien sin ajustar 30 parámetros.

Mobile fallback obligatorio; la experiencia no puede “romperse silenciosamente”.

Reduced motion y pausa forman parte del producto, no de una auditoría posterior.

El cliente final recibe viewer, nunca editor.

Cada feature debe mapearse a activación, retención, monetización o reducción de riesgo.

5. Modelo de negocio y pricing

5.1 Decisión: catálogo de lanzamiento vs catálogo objetivo

La versión 1.0 proponía Free, Creator, Pro, Studio y Enterprise. La estructura es válida a largo plazo, pero cinco planes desde el primer pago multiplican precios, webhooks, soporte, QA y confusión. Se recomienda:

Catálogo de lanzamiento

Free/Demo: adquisición y prueba.

Pro: plan autoservicio principal.

Studio: plan para agencias/equipos; puede iniciarse sales-assisted.

Enterprise: contacto comercial, no autoservicio.

Experimento posterior

Creator (€19): sólo se activa si entrevistas y datos muestran una barrera real entre Free y Pro. No debe construirse por intuición.

Los precios de referencia de la versión 1.0 —Pro 49 €, Studio 129 €, Enterprise desde 499 €— se conservan como hipótesis iniciales, no como verdad. Spline confirma un patrón de free limitado, export sin watermark, video/code export, créditos de IA y Enterprise con SSO, version history y soporte [M1]. Webflow confirma que publishing, governance, permisos, soporte y escala son unidades de valor, no sólo la edición [M2].

5.2 Fuente de verdad comercial

Los planes no se codifican como condicionales dispersos. Deben existir:

products y prices configurables;

plan_versions inmutables;

features y plan_entitlements;

workspace_subscriptions;

entitlement_snapshots;

overrides comerciales auditados;

fecha de vigencia y grandfathering.

El frontend muestra la proyección recibida de /v1/entitlements; no interpreta directamente estados de Stripe.

5.3 Dimensiones monetizables

Acceso a efectos/módulos premium.

Número de seats.

Proyectos activos y versiones retenidas.

Storage y egress.

Publicaciones/embeds activos.

Export premium y render server-side.

AI runs.

Review links y colaboración.

Soporte, SLA, SSO, dominios y governance.

5.4 Créditos: recomendación revisada

No mezclar tres conceptos:

Quota: límite incluido por plan.

Credit: unidad comercial prepago para uso variable.

COGS event: coste real medido por proveedor/worker/storage.

La tabla de créditos v1.0 es un punto de partida, pero no debe fijarse sin benchmarking de coste. El sistema interno debe utilizar enteros y un ledger append-only. Flujo de una acción costosa:

estimar coste;

reservar créditos;

ejecutar;

capturar al completar;

liberar al fallar/cancelar;

registrar coste real y margen.

Stripe recomienda una plataforma específica cuando existen prepaid credits, burndown, contratos enterprise, pricing dimensional o visibilidad de uso en tiempo real [O2]. Para la paid beta, Escaparates Pro debe mantener su ledger interno y vender packs mediante pagos puntuales; no adoptar una plataforma metered compleja hasta que el volumen lo justifique.

5.5 Unit economics

Métricas mínimas:

ARPA y MRR neto.

Gross revenue retention y logo churn.

CAC y CAC payback.

Contribution margin por plan.

COGS por export, publicación, GB y AI run.

Payment failure recovery.

Support minutes por workspace.

Fórmulas operativas:

Contribution Margin = Revenue - payment fees - storage/egress - compute - AI - variable support

Net New MRR = New MRR + Expansion MRR - Contraction MRR - Churned MRR

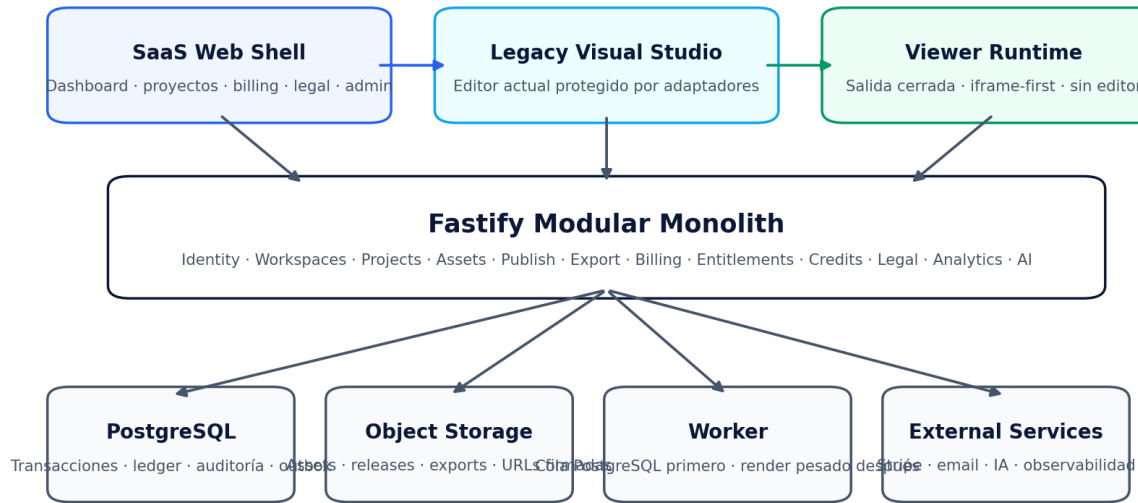
Activation Rate = workspaces with first value / verified new workspaces

Publish Success Rate = successful releases / publish attempts

Los escenarios MRR del documento v1.0 no deben presentarse como forecast. Deben convertirse en un modelo editable con inputs observados.

6. Arquitectura objetivo

Arquitectura objetivo: monolito modular + runtime visual protegido



Principio: contratos versionados y límites server-side; ningún proveedor externo es la fuente de autorización en tiempo real.

Arquitectura objetivo

6.1 Decisiones arquitectónicas

ADR-001 — Monolito modular primero

Mantener una única API desplegable mientras el volumen sea bajo. Separar dominios por módulos internos, transacciones y contratos; no por servicios de red. Crear un worker independiente sólo para tareas pesadas.

ADR-002 — Proteger el runtime visual

No reescribir el editor. Introducir un anti-corruption layer:

ProjectDocumentAdapter traduce el estado legacy al contrato canónico.

EffectManifestRegistry describe capacidades y controles.

ViewerCompiler produce salida cerrada.

feature flags permiten migración incremental.

ADR-003 — Contratos versionados

ProjectDocument, EffectManifest, ViewerRelease, CampaignPlan y eventos de uso tienen schemaVersion. Cada cambio incompatible requiere migrador y fixtures.

ADR-004 — Same-site application boundary

Preferencia de producción:

app.escaparatespro.com para frontend;

api.escaparatespro.com o proxy /api same-site;

cookies Secure, HttpOnly, SameSite=Lax cuando sea viable;

CSRF defense para mutaciones basadas en cookie.

ADR-005 — Object storage privado por defecto

La base de datos guarda metadata e IDs. Assets, releases y artifacts viven en storage S3-compatible con URLs firmadas cortas.

ADR-006 — Iframe-first embed

El iframe es la primera integración pública porque proporciona aislamiento, revocación y CSP. El script loader llega después y sólo monta un iframe/config validada.

ADR-007 — Postgres queue antes de Redis

Para el primer worker, usar una cola transaccional respaldada por PostgreSQL y outbox. Redis se introduce sólo si métricas demuestran bloqueo o throughput insuficiente.

6.2 Estructura de módulos backend

```
apps/api/src/
modules/
  identity/
  workspaces/
  projects/
  presets/
  assets/
  publishing/
  embeds/
  exports/
  billing/
  entitlements/
  credits/
  legal/
  analytics/
  notifications/
  ai/
  admin/
platform/
  database/
  storage/
  jobs/
  observability/
  security/
shared/
  errors/
  ids/
  validation/
  time/
```

No realizar un refactor masivo. Cada nueva microfase mueve sólo el dominio tocado.

6.3 Estados de entorno

Local

DB local/efímera; Stripe test; bucket dev; secretos locales

Implementación y tests rápidos

Preview

Datos aislados o mocks; Stripe test; storage con prefijo por PR; acceso de equipo

QA visual y de contrato por pull request

Staging

DB y bucket persistentes separados; Stripe test; webhooks propios; acceso privado

Integración end-to-end y ensayos operativos

Production

DB, storage, secretos y Stripe live exclusivos; dominio final; backups y alertas

Servicio real para clientes

Nunca compartir secretos, webhooks o buckets entre staging y producción.

7. Contratos canónicos

7.1 ProjectDocument

Es la unidad de persistencia y compilación. El schema completo se entrega en schemas/project-document.schema.json.

Principios:

asset references por UUID, nunca URLs firmadas;

module ID + version + manifest hash;

controles libres pero validados contra controlsSchema del manifest;

output y accessibility explícitos;

revision para optimistic locking;

provenance/licencias y AI run IDs;

límite de tamaño del documento;

migradores puros y testeados.

Ejemplo reducido:

```
{
  "schemaVersion": "1.0.0",
  "projectId": "6f3e8ab6-87d2-42e7-86d0-ff68e7a37f28",
  "workspaceId": "3a0db366-79cf-4fa3-88e5-f1d8de83858c",
  "name": "Hero campaña verano",
  "status": "draft",
  "revision": 12,
  "mode": "website-module",
  "module": {
    "id": "rgb-motion-triptych-pro",
    "version": "1.1.0",
    "kind": "website-module",
    "manifestHash": "sha256:..."
  },
  "assets": [
    { "slotId": "primary", "assetId": "1e74...", "variant": "web-optimized" }
  ],
  "controls": { "intensity": 0.72, "speed": 0.55 },
  "branding": { "headline": "Nueva colección", "cta": { "label": "Descubrir", "url": "https://example.com" } },
  "output": { "preset": "web-hero-16:9", "width": 1920, "height": 1080, "fps": 60, "quality": "high" },
  "accessibility": { "reducedMotion": "auto", "keyboardNavigation": true, "pauseControl": true }
}
```

7.2 EffectManifest

Todo efecto/módulo debe declarar:

identidad y versión;

estado de madurez;

entry point e isolation mode;

media slots;

control schema y defaults;

compatibilidad desktop/tablet/mobile;

reduced motion;

formatos exportables;

entitlement keys;
performance tier;
licencia, fuente y notice;
migraciones.

Esto sustituye heurísticas como “premium si el tag contiene premium” y permite gobernar el catálogo.

7.3 ViewerRelease

Objeto inmutable:

```
releaseId, publishId, projectVersionId, compilerVersion,  
manifestVersions, contentHash, artifactPrefix, cspPolicy,  
allowedOrigins, status, createdAt, revokedAt
```

La URL estable apunta a un release activo; no se sobrescriben artifacts históricos.

7.4 CampaignPlan de IA

La IA no devuelve código. Devuelve JSON:

```
briefSummary, audience, objective, visualDirection,  
recommendedModules[], copyVariants[], channelVariants[],  
assetAssignments[], outputPresets[], estimatedCredits,  
risks[], confidence, modelMetadata
```

Un compilador determinista lo transforma en uno o varios ProjectDocument. El usuario aprueba antes de publicar o consumir acciones caras.

8. Diseño de API

8.1 Estándar

Base /v1.

OpenAPI generado desde schemas de runtime.

IDs UUID opacos.

UTC ISO-8601.

Cursor pagination.

Idempotency-Key en mutaciones con coste o efecto externo.

If-Match/revision para updates de proyecto.

errores estables y no filtrado de secretos.

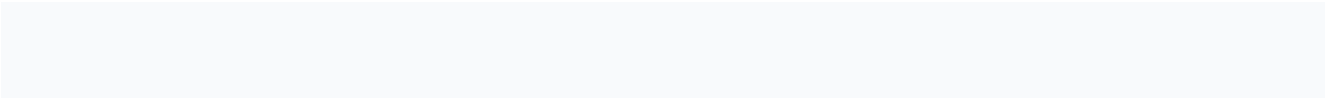
Respuesta objetivo:

```
{
  "data": {},
  "meta": {"requestId": "..."}
}
```

Error objetivo:

```
{
  "error": {
    "code": "PROJECT_REVISION_CONFLICT",
    "message": "The project changed since it was loaded",
    "details": {"expectedRevision": 12, "actualRevision": 13},
    "requestId": "..."}
}
```





8.2 Endpoint matrix prioritaria

Auth
POST /v1/auth/register
Actual
Añadir verification state
Auth
POST /v1/auth/login
Actual
Mover sesión a cookie segura
Auth
POST /v1/auth/refresh
G1
Rotación y reuse detection
Auth
POST /v1/auth/forgot-password
G1
Respuesta no enumerable
Auth
POST /v1/auth/reset-password
G1
Revocar sesiones
Auth
GET /v1/auth/sessions
G1
Gestión por dispositivo

Workspaces

GET /v1/workspaces

G2/G5

Membership-aware

Workspaces

POST /v1/workspaces/:id/select

G2

Contexto explícito

Entitlements

GET /v1/entitlements

G1

Snapshot canónico

Projects

POST /v1/projects

G2

Valida ProjectDocument

Projects

GET /v1/projects/:id

G2

Tenant scoped

Projects

PATCH /v1/projects/:id

G2

Optimistic locking

Versions

POST /v1/projects/:id/versions

G2

Snapshot immutable

Presets

POST /v1/presets/import-legacy

G2

Idempotente

Assets

POST /v1/assets/upload-intents

G3

Quota + signed URL

Assets

POST /v1/assets/:id/complete

G3

Validación async

Publish

POST /v1/publishes

G3

Compila release

Publish

POST /v1/publishes/:id/revoke

G3
 Auditado
 Exports
 POST /v1/exports
 G4
 Reserva créditos
 Credits
 GET /v1/credits/balance
 G4
 Ledger projection
 Billing
 POST /v1/billing/checkout
 Actual/G1
 URL server-side
 Billing
 POST /v1/billing/portal
 G1
 Stripe portal
 Billing
 POST /v1/webhooks/stripe
 G1
 Raw body, signature
 Legal
 GET /v1/legal/documents/current
 G1
 Versiones vigentes
 Legal
 POST /v1/legal/acceptances
 G1
 Before paid action
 AI
 POST /v1/ai/campaign-plans
 G6
 Structured output

9. Identidad, sesión y seguridad

9.1 Estado actual

La API crea tokens opacos, guarda sólo su hash y aplica expiración [G10]. Esta decisión es correcta. El problema es el transporte y almacenamiento: el cliente guarda el token largo en localStorage y lo usa como bearer para todas las llamadas [G4].

9.2 Arquitectura de sesión objetivo

Opción preferida:

cookie de sesión/refresh Secure; HttpOnly; SameSite=Lax; Path=/;

access token corto sólo en memoria si se necesita;

rotation en cada refresh;

token family y reuse detection;

revocación por sesión y global;

CSRF token/origin checks en mutaciones;

dominio same-site;

session metadata: device, IP truncada, user agent, last used;

expiry absoluta e idle expiry.

9.3 Password y recuperación

El script nativo actual es aceptable como foundation, pero debe versionarse con parámetros explícitos y capacidad de rehash. Añadir:

email verification;

reset token one-time y corto;

invalidación de sesiones al cambiar password;

rate limits específicos para register/login/reset;

protección contra enumeración;

bloqueo progresivo, no bloqueo permanente fácil de abusar;

MFA sólo cuando haya necesidad enterprise o cuentas administrativas.

9.4 Multi-tenancy y RBAC

Roles objetivo:

owner;

admin;

editor;

viewer;

billing.

Cada route debe resolver:

principal → selected workspace → membership → role → entitlement → quota → resource ownership

No aceptar workspaceId sin comprobar membresía. Añadir tests de IDOR y tenant leakage. PostgreSQL RLS puede adoptarse como defensa en profundidad más adelante; no sustituye autorización de aplicación.

9.5 Baseline OWASP

Usar OWASP ASVS como catálogo de verificación y el File Upload Cheat Sheet para assets [O3, O4]. Paid beta: objetivo ASVS Level 1 completo y selección Level 2 para auth, access control, session, files, crypto y logging.

10. Persistencia de proyectos y presets

10.1 Modelo de guardado

projects contiene metadata, estado, revision y puntero a versión actual.

project_current_documents o snapshot actual optimizado para autosave.

project_versions es inmutable.

autosave con debounce y backoff;

revision evita lost updates;

versionar en hitos: primer guardado, cambio de módulo, publicación, aprobación y restore;

soft delete con retention configurable;

restore crea nueva versión; no reescribe historia.

10.2 Integración sin romper editor

Capturar estado actual mediante adapter.

Validar contra ProjectDocument.

Guardar en backend detrás de feature flag.

Mantener fallback local temporal.

Comparar round-trip local → cloud → editor con fixtures.

Activar por cohortes.

Retirar legacy sólo tras telemetría y rollback window.

10.3 Presets

ep_presets_v1 debe migrarse una vez mediante fingerprint. El backend devuelve presets cloud y conserva el original local hasta confirmación. No borrar ni sobrescribir silenciosamente.

11. Assets y storage

11.1 Pipeline

upload-intent: valida auth, plan, quota, tipo declarado y tamaño.

Crea asset con estado uploading.

Emite URL firmada de corta duración.

Cliente sube directamente a quarantine prefix.

complete: confirma objeto y dispara validación.

Detecta magic bytes, dimensiones, duración, codec y metadata.

Genera variantes/thumbnail/poster.

Marca ready, rejected o quarantined.

11.2 Reglas

Private by default.

Nombres internos aleatorios.

No confiar en extensión ni Content-Type del navegador.

Allowlist explícita.

Límites por bytes, pixels, duración y frames.

Nunca servir uploads desde el mismo origen ejecutable que la app sin controles.

Re-encode de imágenes cuando sea viable.

Video servido con content type correcto y range support.

URLs firmadas no se guardan en ProjectDocument.

Borrado lógico, grace period y garbage collector reference-aware.

Data export/delete conforme a privacidad.

12. Publicación, embed y export

Pipeline determinista de publicación y embed



Pipeline de publicación

12.1 Publish compiler

El compilador recibe una versión inmutable de proyecto y produce:

- index.html mínimo;
- runtime versionado;
- config serializada y validada;
- referencias a assets publicables;
- CSP;
- metadata social opcional;
- fallback móvil/reduced motion;
- manifest de artifacts con hashes.

No incluir paneles, librería completa, secrets, endpoints internos o rutas de edición.

12.2 Stable URL

- publishId opaco.
- alias estable a currentReleaseId.
- releases inmutables.
- rollback instantáneo cambiando puntero.
- revocación y expiración.
- dominio custom como feature futura.

12.3 Embed

- Primero iframe:
- sandbox mínimo necesario;
- CSP estricta;
- allowedOrigins opcional;

protocolo postMessage versionado;
 resize seguro;
 eventos de ready, CTA y error;
 sin acceso al editor.

Script embed futuro: loader pequeño que crea el iframe. No exportar todo el runtime como script global sin aislamiento.

12.4 Export

Mantener export local para previews y formatos ligeros. Registrar cada export en backend incluso si se ejecuta localmente. Mover a worker sólo:

MP4/2K/4K;
 secuencias largas;
 tareas que superen umbral de tiempo/memoria;
 jobs que requieran reproducibilidad y garantía.

Estados:

requested → validating → reserved → queued → running → succeeded
 failed / cancelled / expired

13. Billing, entitlements y créditos

Flujo correcto de billing y entitlements



Billing y entitlements

13.1 Principio

Stripe informa pagos; PostgreSQL proyecta el acceso. La autoridad de una request es el snapshot interno actualizado por eventos verificados.

Stripe documenta que las suscripciones cambian de forma asíncrona, deben gestionarse con webhooks verificados y que eventos como `invoice.paid`, `invoice.payment_failed` y `customer.subscription.updated/deleted` determinan provisión o revocación [O1].

13.2 Tablas mínimas

billing_customers;
 subscriptions;
 subscription_items;
 webhook_events;
 plan_versions;
 features;
 plan_entitlements;
 entitlement_snapshots;
 credit_accounts;
 credit_transactions;
 credit_reservations;
 billing_reconciliation_runs.

13.3 Webhook processing

leer raw body;
 verificar firma y timestamp;
 insertar `webhook_events` con unique provider event ID;
 responder rápido tras persistencia o procesar transaccionalmente;

handler idempotente;
 actualizar projection;
 emitir outbox event;
 recomputar entitlements;
 registrar resultado/error;
 retry con backoff y dead-letter operativo.

13.4 Estados internos

Mapear Stripe a estados de producto explícitos:

free;
 trialing;
 active;
 grace;
 past_due_limited;
 paused;
 canceled_until_period_end;
 canceled;
 blocked.

La política de gracia debe ser producto, no un if improvisado.

13.5 Reconciliación

Job diario:
 compara customers/subscriptions/precios;
 detecta webhooks perdidos;
 repara projections;
 alerta discrepancias;
 no revoca automáticamente casos ambiguos sin política.

14. AI Campaign Builder

14.1 Diferenciador correcto

No chatbot genérico. Debe actuar como director creativo estructurado:

recibe brief y assets;

clasifica objetivo/canal;

recomienda módulos compatibles;

genera copy y dirección visual;

crea tres CampaignPlans;

estima créditos;

compila variantes editables;

el usuario aprueba.

14.2 Arquitectura

provider abstraction;

prompt registry versionado;

structured output con schema;

policy layer;

cost budget y timeout;

redacción de PII/secrets;

no entrenamiento con datos de cliente por defecto;

retention configurable;

model/prompt/input hash y provenance;

evaluation dataset;

fallback determinista sin IA.

14.3 No-go

El modelo no escribe HTML/JS ejecutable en producción.

No navega URLs de usuario desde la red interna sin sandbox y allowlist.

No publica ni cobra sin confirmación.

No usa assets del cliente para entrenar sin opt-in contractual.

No promete copyright sobre outputs más allá de contratos de proveedor y ley aplicable.

14.4 AI Act y transparencia

El AI Director creativo no parece, por su caso de uso previsto, un sistema high-risk típico; esta clasificación debe validarse jurídicamente y revisarse si se amplía a decisiones sobre personas. Mantener inventario de modelos, finalidad, datos, proveedores, transparencia, human oversight y logs. El Reglamento (UE) 2024/1689 aplica obligaciones por fases y debe tratarse como requisito vivo [O6].

15. Legal, compliance y trust readiness

Esta sección es una especificación de producto y datos, no asesoramiento jurídico. Los textos iniciales pueden redactarse por comparación estructural, pero no copiarse literalmente de competidores y deben validarse por un abogado antes del lanzamiento público.

15.1 Legal & Trust Center mínimo

Aviso legal / identificación del prestador.

Términos de servicio SaaS.

Política de privacidad.

Política de cookies.

CMP real con aceptar/rechazar/configurar y revocación.

Billing, renovación, cancelación y reembolsos.

Data Processing Addendum.

Lista de subprocessors.

Acceptable Use Policy.

Propiedad intelectual, licencia de contenido y garantías del usuario.

Política de IA y tratamiento de inputs/outputs.

Copyright/takedown y canal de reclamaciones.

Accessibility statement.

Security disclosure policy.

Status page y soporte.

La LSSI exige que la información identificativa sea accesible de forma permanente, fácil, directa y gratuita; también regula información contractual y cookies. La normativa de consumidores exige información precontractual y reglas específicas para contenido/servicios digitales cuando la relación sea B2C. La guía de cookies de AEPD exige opciones reales y retirada del consentimiento [L1–L4].

15.2 B2B-first

Recomendación comercial: lanzar como herramienta de uso profesional/B2B y describirlo claramente. Esto reduce complejidad, pero no elimina privacidad, cookies, facturación ni derechos que puedan aplicar a autónomos/consumidores. El checkout debe recoger país, tipo de cliente y datos fiscales suficientes.

15.3 Aceptación versionada

Tablas:

`legal_documents(id, type, version, effective_at, content_hash, url, status);`

`legal_acceptances(user_id, workspace_id, document_id, accepted_at, ip, user_agent, source);`

`consent_records(subject_id, purpose, status, captured_at, withdrawn_at, evidence).`

Antes del primer pago: aceptar TOS y privacy vigentes. No usar checkbox premarcada. Guardar hash y versión.

15.4 Privacidad

data map y RoPA simplificado;

propósito/base legal por evento;

minimización;

retention schedule;
 DSR export/delete;
 encryption in transit/at rest;
 vendor due diligence;
 transfer mechanism;
 breach runbook;
 analytics/cookies separados de operación esencial.

15.5 IP y provenance

Cada módulo/asset de referencia debe registrar:

autor/origen;
 URL y commit/tag;
 licencia SPDX;
 adaptación realizada;
 notices;
 restricciones de assets;
 fecha de revisión.

Crear THIRD_PARTY_NOTICES.md. No asumir que una interacción pública permite copiar código o assets.
 Para el repositorio público, decidir explícitamente:

privado + propietario;
 open-core con paquetes delimitados;
 o licencia pública consciente.

La ausencia de decisión es un riesgo comercial.

16. Accesibilidad y calidad visual

Objetivo: WCAG 2.2 AA para shell, auth, dashboard, billing, legal, review y controles esenciales [O5]. Los viewers creativos deben ofrecer:

reduced motion;

pausa/replay;

keyboard path cuando haya interacción;

texto alternativo/contexto;

contraste;

foco visible;

fallback cuando WebGL/WebGPU no esté disponible;

no flashes peligrosos;

target size adecuado;

autenticación accesible.

QA visual obligatorio en:

1440×900 desktop;

1024×768 tablet;

390×844 móvil;

touch y teclado;

reduced motion;

conexión lenta y GPU baja.

17. Observabilidad, SRE y operaciones

17.1 Telemetría

Logs estructurados con:

```
requestId, traceId, userId hash, workspaceId, route,
status, durationMs, errorCode, deployment, release
```

Nunca password, tokens, Stripe secrets, full signed URLs ni contenido sensible.

Métricas:

API RPS, p50/p95/p99 y errores;

auth success/failure/throttle;

DB pool y slow queries;

webhook lag/failure/retry;

publish compile duration/failure;

export queue depth/runtime/cost;

asset rejection/storage/egress;

AI latency/tokens/cost/schema failure;

entitlement denials;

payment recovery.

17.2 SLOs paid beta

API availability

99,5% mensual

Auth success p95

< 800 ms excluyendo red externa

Project read p95

< 400 ms

Project autosave p95

< 700 ms

Publish success

> 98% para módulos soportados

Webhook processed p95

< 60 s

Asset validation success

> 97% para inputs permitidos

Restore test

mensual

No vender SLA enterprise hasta medir al menos 90 días.

17.3 Runbooks

API down;

DB unavailable;

migration failure;

Stripe webhook backlog;

accidental plan grant/revoke;
storage unavailable;
publish corruption;
credential compromise;
data deletion request;
incident communication.

18. CI/CD y gobierno de GitHub

18.1 Estado y objetivo

La CI actual no ejecuta tests aunque el paquete dispone de ellos [G9]. Debe convertirse en required checks:

install reproducible;

format/lint;

typecheck;

unit tests;

integration tests con PostgreSQL;

build;

migration apply from empty y previous snapshot;

OpenAPI/schema drift;

frontend syntax/build;

Playwright smoke;

visual regression de rutas críticas;

CodeQL;

dependency review;

secret scan;

license/notice check;

artifact/SBOM cuando proceda.

GitHub ofrece CodeQL para análisis semántico y dependency graph para visibilidad de supply chain [S1, S2].

18.2 Archivos de gobierno

SECURITY.md;

CONTRIBUTING.md;

CODEOWNERS;

PULL_REQUEST_TEMPLATE.md;

THIRD_PARTY_NOTICES.md;

decisión de LICENSE;

docs/status/current-state.md;

docs/adr/;

docs/runbooks/;

AGENTS.md como fuente común para Codex/Claude.

18.3 Branch protection

no push directo a master;

checks obligatorios;

review independiente para P0;

conversation resolution;

stale branch update;

merge commit/squash definido consistentemente;
deployment preview antes de merge frontend.

19. Analytics y experimentación

19.1 Taxonomía de eventos

Eventos mínimos:

```
account_registered
email_verified
workspace_created
project_created
project_first_value_completed
asset_upload_started/completed/rejected
preview_completed
export_requested/succeeded/failed
publish_requested/succeeded/revoked
embed_loaded
cta_clicked
checkout_started/completed
subscription_activated/past_due/canceled
credits_reserved/captured/released
ai_plan_requested/succeeded/rejected
```

Todos incluyen event version, workspace, plan snapshot, project/module IDs y client/server timestamp cuando corresponda. Evitar PII en propiedades analíticas.

19.2 Funnel

Visit → register.

Register → verified.

Verified → project created.

Project → first value.

First value → checkout.

Checkout → active.

Active → second week value.

Paid → retained day 30/90.

19.3 Experimentos prioritarios

CTA de upgrade tras preview vs tras export.

Founder offer vs precio estándar.

Template por sector vs biblioteca abierta.

Guided onboarding vs editor vacío.

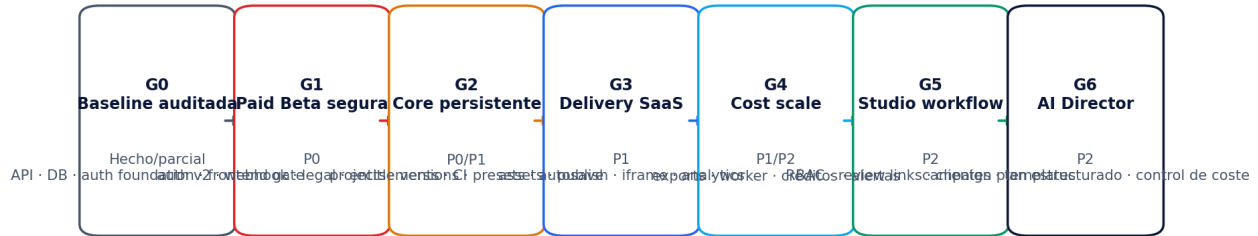
Pro 39/49 € con entrevistas y cohortes, no sólo A/B de bajo tráfico.

Existencia del plan Creator.

Cada experimento define hipótesis, métrica, guardrail, muestra mínima razonable y decisión.

20. Roadmap por release gates

Release gates — del prototipo avanzado al SaaS comercial



Paid beta realista: G1 + mínimo de G2. Lanzamiento público: G1-G3. Enterprise comercial: después de G5 y evidencia de demanda.

Estimación orientativa para un ejecutor principal asistido por IA: 81-135 días de ingeniería hasta G6; paid beta enfocada en 16-25 días.

Release gates

20.1 G0 — Baseline auditada

Estado: completado/parcial.

Incluye API, DB, auth foundation, default workspace, Checkout foundation y frontend auth gate. No debe confundirse con paid beta.

20.2 G1 — Paid Beta segura

Objetivo: poder cobrar a un grupo cerrado sin depender de acciones manuales inseguras.

Entregables:

- auth v2 segura;
- Stripe webhook + portal + reconciliación;
- entitlements server-side;
- checkout URL allowlist;
- Legal & Trust Center;
- CI required checks;
- dominios y entornos;
- backups/alerts/runbooks;
- licensing decision.

Exit criteria: ningún acceso Pro se activa desde navegador; no existe token largo legible por JS; pago/impago/cancelación se refleja; términos aceptados; tests y alertas pasan.

Estimación: 16-25 días de ingeniería focalizada, más validación legal externa.

20.3 G2 — Core persistente

- ProjectDocument v1;
- CRUD/autosave/conflict;

versiones/restore;
presets cloud e import legacy;
dashboard mínimo de proyectos.

Exit: cerrar navegador y recuperar exactamente el proyecto en otro dispositivo.

20.4 G3 — Delivery SaaS

object storage;
asset validation;
ViewerRelease;
stable publish/revoke;
iframe embed;
analytics básicos.

Exit: un cliente externo puede abrir una URL estable sin editor y el propietario puede revocarla.

20.5 G4 — Cost scale

export registry;
credit ledger;
worker sólo si lo exige la demanda;
notificaciones y cost telemetry.

Trigger worker: fallos/tiempos del navegador, demanda de formatos premium o coste que requiera control central.

20.6 G5 — Studio workflow

RBAC completo;
invites;
client folders;
review links;
aprobación y comentarios;
templates compartidas.

Trigger: al menos cinco agencias usando más de un cliente/proyecto recurrente.

20.7 G6 — AI Director

CampaignPlan;
provider abstraction;
prompt registry;
evaluation;
tres variantes compilables;
credit estimates.

Trigger: core estable y dataset de al menos 50 briefs/resultados revisados.

20.8 G7 — Enterprise

SSO, SCIM, custom domains, SLA y governance sólo con demanda firmada o tres oportunidades cualificadas. “Enterprise-grade” describe cómo se construye desde G1; no obliga a construir Enterprise antes de product-market evidence.

20.9 No construir todavía

PayPal subscriptions;
 marketplace;
 white-label completo;
 public API;
 SSO/SCIM;
 microservicios;
 Redis por defecto;
 4K ilimitado;
 colaboración en tiempo real;
 AI que genera código;
 apps móviles nativas.

21. Definition of Ready y Definition of Done

21.1 Ready

Una microfase está lista cuando posee:

- objetivo de producto único;
- baseline SHA;
- dependencias cerradas;
- paths permitidos/prohibidos;
- contratos y estados;
- criterios de aceptación observables;
- tests positivos y negativos;
- plan de migración/rollback;
- owner y revisor.

21.2 Done

- rama real agent/<epic>-<slug>;
- commits intencionales;
- PR con scope y riesgos;
- tests requeridos en CI;
- no errores de typecheck/build;
- migration y rollback strategy;
- OpenAPI/schemas/docs actualizados;
- auth/tenant/plan negative tests;
- logs/metrics para el nuevo flujo;
- QA visual desktop/tablet/mobile;
- preview verificada;
- review independiente resuelta;
- no secretos ni notices pendientes.

22. Protocolo obligatorio para Codex y Claude

22.1 División de responsabilidades

Codex

implementar;
 escribir migraciones y tests;
 ejecutar QA;
 corregir CI;
 preparar PR y evidencia.

Claude

revisar arquitectura, seguridad, legal-tech y coherencia;
 comparar implementación con spec;
 identificar edge cases;
 revisar diff y tests;
 emitir comentarios accionables.

Railway

deploy, variables, Postgres, networking, logs, metrics y healthchecks;
 no usar como agente de razonamiento prolongado.

GitHub/Vercel

GitHub es la fuente verificable de cambios;
 Vercel preview es evidencia visual antes de integración.

22.2 Regla de entrega

Para tareas de código, **no aceptar código pegado en chat ni ZIP como sustituto**. La entrega válida incluye rama, commit, push, PR, logs, preview y QA visual. Los módulos standalone deben abrirse y revisarse visualmente antes de integrarse.

22.3 Prompt operativo maestro

Actúa como Principal Engineer de Escaparates Pro.

Repositorio: Juanmaes83/escaparates-pro

Rama base: master

Antes de cambiar nada:

1. identifica el SHA de master;
2. lee Enterprise Master Specification v2, current-state, ADRs y contratos;
3. confirma que el working tree está limpio;
4. crea agent/<epic-id>-<slug>.

Implementa una sola microfase. No amplíes scope ni reescribas el editor.

Todo recurso SaaS pertenece a un workspace. Autorización, cuotas y entitlements son server-side.

Los pagos sólo cambian acceso por webhooks verificados e idempotentes.

No guardes refresh tokens o secretos legibles por JavaScript.

No publiques el editor, URLs firmadas persistentes ni JavaScript arbitrario de usuario.

No ejecute migraciones automáticamente en start.

Entrega mediante GitHub real: rama, commits, push y PR. Incluye tests, migraciones,

OpenAPI/schemas, documentación, logs, preview y QA visual desktop/tablet/móvil.

No uses texto o ZIP como sustituto de la implementación.

Detalla:

- objetivo y no-goals;
- archivos modificados;
- decisiones y trade-offs;
- tests positivos/negativos;
- seguridad/tenant/plan;
- deploy y rollback;
- riesgos residuales;
- siguiente microfase recomendada.

La plantilla completa se entrega en templates/agent-task-template.md.

23. Modelo de datos objetivo



23.1 Identidad y tenant

- users
- Identidad
 - email normalizado unique, verification state
- credentials
- Password/providers
- hash versionado, no secretos en claro
- sessions
- Sesiones
- token family, hash, rotation, revoked_at
- workspaces
- Tenant
 - slug unique, owner no equivale a único acceso
- workspace_members
- Membresía
 - role enum, status, invited_by
- workspace_invites
- Invitaciones
 - token hash, expiry, single use



23.2 Producto

- projects
 - Metadata y current revision
 - workspace scoped, soft delete
- project_documents
 - Snapshot actual
 - schema_version, revision unique
- project_versions
 - Historia inmutable
 - (project_id, version_number) unique
- presets
 - Config reusable
 - workspace/public/provenance
- module_manifests
 - Registry
 - (module_id, version) unique, content hash
- templates
 - Duplicables
 - source version y rights



23.3 Assets y delivery

- assets
- Metadata/lifecycle
 - workspace, status, storage key unique
- asset_variants
- Thumbnails/transcodes
 - source asset, profile unique
- asset_usages
- Referencias
 - evita borrar asset usado
- publishes
 - URL estable
 - current release, revoked_at
- publish_releases
- Release immutable
 - content hash unique por publish
- embed_policies
- Origins/sandbox
 - policy versionada
- exports
- Solicitud e historial

idempotency, entitlement snapshot
export_jobs
Worker state
retry, lease, timeout
export_artifacts
Files
checksum, retention, size



23.4 Billing y control

- plan_versions
- Catálogo comercial
- immutable tras vigencia
- features
- Feature keys
- stable semantic keys
- plan_entitlements
- Quotas/features
- typed values
- billing_customers
- Mapping proveedor
- provider/customer unique
- subscriptions
- Projection
- provider subscription unique

webhook_events

Inbox idempotente

provider event unique, raw hash

entitlement_snapshots

Acceso efectivo

version, source, computed_at

credit_accounts

Cuenta por workspace

una currency/unidad

credit_transactions

Ledger

append-only, idempotency unique

credit_reservations

Reserva

expiry y terminal state

reconciliation_runs

Control

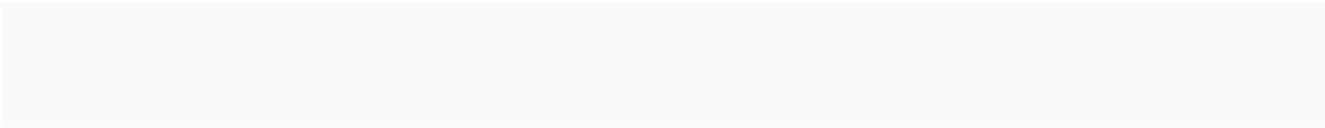
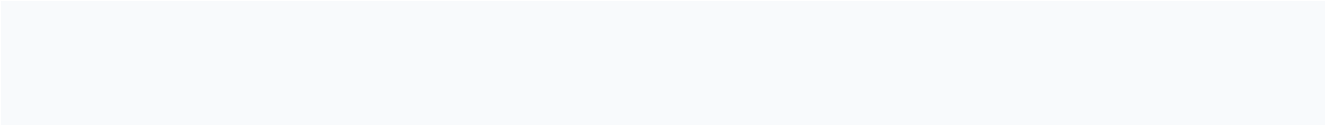
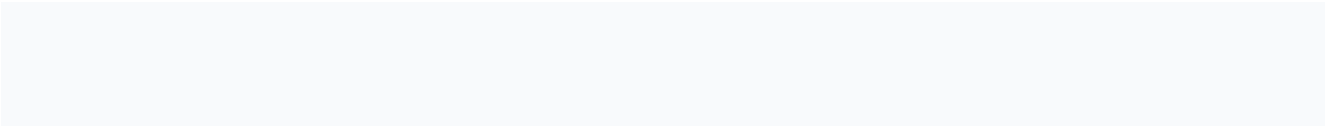
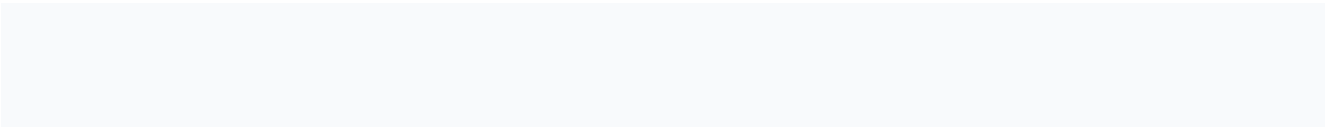
discrepancy counts



23.5 Trust, analytics y AI

- legal_documents / legal_acceptances
- Contratos versionados y evidencia
- consent_records
- Consentimiento por propósito
- audit_logs
- Acciones administrativas y de producto
- security_events
- Auth, abuse y incidentes
- usage_events
- Eventos append-only
- usage_rollups
- Dashboards/cuotas
- outbox_events
- Integración fiable post-transaction
- notification_deliveries
- Email/in-app lifecycle

ai_runs
Provider/model/prompt/cost/status
prompt_versions
Prompts versionados/evaluados
ai_artifacts
CampaignPlans y outputs estructurados



24. Riesgos principales

XSS roba sesión — Media

Crítico. Scripts o HTML dinámico con token largo accesible al navegador

Auth v2, CSP, aislamiento iframe y eliminación de refresh token en localStorage

Pago y acceso divergen — Alta

Crítico. Tickets “he pagado y no tengo Pro” o acceso tras impago

Webhooks verificados, projection interna y reconciliación diaria

Tenant leakage — Media

Crítico. Acceso por IDs directos o helpers basados sólo en owner

Middleware de workspace/membership y tests IDOR negativos

Editor se rompe al integrar — Alta

Alto. PRs extensas sobre scripts críticos y regresiones visuales

Adapters, feature flags, fixtures y visual regression

Coste 4K/IA destruye margen — Media

Alto. Uso costoso sin metering o estimación

Reserva/captura de créditos y telemetría de coste real

Publicación incluye editor — Media

Alto. Bundle enorme, rutas internas o secretos en artifacts

Compiler por allowlist y auditoría automática del release

Upload malicioso — Media

Alto. Se confía en extensión o MIME del cliente

Quarantine, magic bytes, límites y re-encode/transcode seguro

Docs contradicen código — Alta

Alto. Agentes repiten fases o implementan contratos obsoletos

Docs-as-code, current-state por commit y contract drift checks

Ambigüedad de licencia — Alta

Alto. Código adaptado sin notice o estrategia pública confusa

Provenance registry, THIRD_PARTY_NOTICES y decisión legal del repo

Producto demasiado amplio — Alta

Alto. Más módulos sin mejorar activación ni retención

Cuña inicial, release gates y no-go list

Churn de uso puntual — Alta

Alto. Export único seguido de cancelación

Projects, publish, review, templates y segundo valor en 14 días

Soporte consume al fundador — Media

Alto. Onboarding manual repetido y tickets previsibles

Onboarding productizado, help center y runbooks

Vendor lock-in IA — Media

Medio. Prompts y outputs acoplados a un proveedor

Provider abstraction, schema y evals portables

Falta de accesibilidad — Media

Medio/alto. Flujos sin teclado, pausa o reduced motion

WCAG gate, QA asistiva y fallback

Migración defectuosa — Media

Alto. Drift, endpoint manual o rollback incierto

Checksums, locks, backup, dry-run y restore drill

25. Launch gates

L0 — Internal

datos ficticios;
no cobros;
pruebas locales/staging.

L1 — Private beta

usuarios invitados;
auth segura;
proyectos mínimos;
términos beta;
soporte directo;
Stripe test.

L2 — Paid beta

G1 completo;
mínimo G2 para guardar/reabrir;
Stripe live;
legal validado;
invoices/tax policy;
backups/alerts;
5–20 clientes.

L3 — Public launch

G1–G3;
onboarding autoservicio;
publicación estable;
status/help;
métricas de activación y soporte.

L4 — Enterprise sales

G5;
audit export;
security pack;
support process;
SLA medible;
DPA/subprocessors maduros.

26. Plan de adquisición de los primeros clientes

Crear 12 demos excelentes, no 20 mediocres: 4 agencias, 3 ecommerce, 2 inmobiliarias, 2 hospitality/retail y 1 estudio creativo.

Mostrar antes/después y tiempo real de creación.

Ofrecer founder cohort limitada: Pro a precio reducido durante 3–6 meses, sin lifetime.

Onboarding concierge para aprender, pero instrumentado.

Entrevista después del primer valor y día 14.

Convertir cada resultado validado en template y caso de uso.

Objetivos de 30 días paid beta:

10 workspaces de pago;

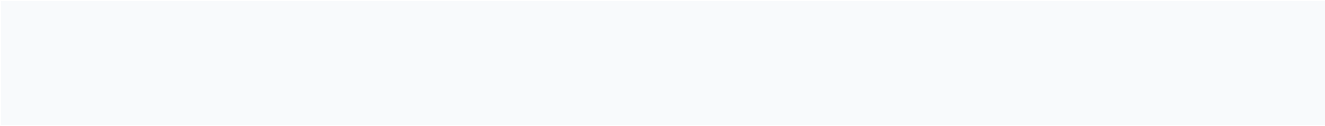
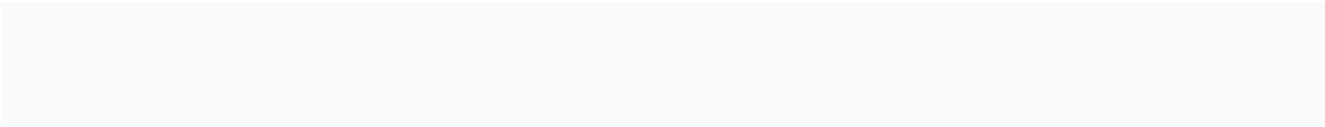
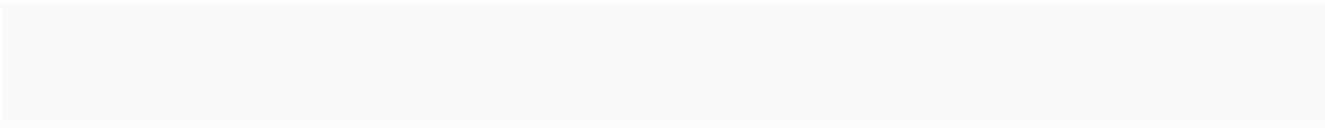
70% activados;

60% consigue segundo valor en 14 días;

5 publicaciones/embeds reales;

menos de 30 min de soporte medio por workspace/semana;

cero incidentes P0 y cero divergencias de billing sin resolver.



27. Backlog de decisiones pendientes

- Dominio final y boundary same-site
- Producto/Ingeniería — G1
- Permite cookies seguras, CORS simple y marca consistente
- Repo private, proprietary u open-core
- Producto/Legal — G1
- Protege el activo y deja notices/licencias sin ambigüedad
- Pro a 39 o 49 €
- Producto — paid beta
- Entrevistas, disposición a pagar y margen esperado

Activar plan Creator
 Producto — después de 50–100 pagos
 Evidencia de un gap real entre Free y Pro
 Storage provider
 Ingeniería — G3
 Coste total, egress, signed URLs, región y DPA
 Email provider
 Ingeniería — G1
 Deliverability, webhooks, DPA y coste
 Analytics vendor o self-host
 Producto/Privacy — G3
 Consentimiento, minimización y coste operativo
 Tecnología del worker
 Ingeniería — G4
 Carga medida y necesidad demostrada
 Proveedor(es) de IA
 Producto/Ingeniería — G6
 Calidad, coste, DPA, retención y portability
 Venta a consumidores
 Producto/Legal — antes de public launch
 Derechos digitales, desistimiento, impuestos y soporte

28. Evidencia y fuentes

Fuentes del proyecto

- [B1] Documento Maestro — Escaparates Pro v1.0, 35 páginas. Objetivo, estrategia, pricing, roadmap y prompt original.
- [G1] GitHub repository metadata: Juanmaes83/escaparates-pro, default branch master, audited 14-07-2026.
- [G2] PR #11, “Frontend Auth Gate + Plan Enforcement”, merged at commit cc3a8002....
- [G3] README.md at audited baseline: product catalog, auth/billing endpoints and local-only publish limitation.
- [G4] js/auth-client.js: ep.refreshToken stored in localStorage and sent as Bearer.
- [G5] js/plan-gate.js: Demo/Free/Pro policies and client-side validation.
- [G6] apps/api/src/routes/billing.ts: single Pro Checkout, optional client URLs, no webhook.
- [G7] apps/api/src/db/schema.ts: current tables and denormalized billing fields.
- [G8] apps/api/src/app.ts: Helmet, CORS, global rate limit and registered routes.
- [G9] .github/workflows/api-ci.yml: typecheck/build without test execution.
- [G10] apps/api/src/routes/auth.ts: register/login/me/logout and session creation.
- [G11] apps/api/src/lib/workspace-defaults.ts: primary workspace by owner and default membership.

Fuentes oficiales y benchmarks (consultadas 14-07-2026)

- [O1] Stripe Docs, “Using webhooks with subscriptions”: <https://docs.stripe.com/billing/subscriptions/webhooks>
- [O2] Stripe Docs, “Basic usage-based billing / Metronome”: <https://docs.stripe.com/billing/subscriptions/usage-based>
- [O3] OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>
- [O4] OWASP File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- [O5] W3C WCAG 2.2: <https://www.w3.org/TR/WCAG22/>
- [O6] Reglamento (UE) 2024/1689 — AI Act: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [L1] BOE, Ley 34/2002 de servicios de la sociedad de la información: <https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758>
- [L2] BOE, Real Decreto Legislativo 1/2007, consumidores y usuarios: <https://www.boe.es/buscar/act.php?id=BOE-A-2007-20555>
- [L3] Reglamento General de Protección de Datos: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [L4] AEPD, Guía sobre el uso de las cookies: <https://www.aepd.es/guias/guia-cookies.pdf>
- [M1] Spline Pricing: <https://spline.design/pricing>
- [M2] Webflow Pricing: <https://webflow.com/pricing>
- [S1] GitHub CodeQL: <https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-cli>
- [S2] GitHub Dependency Graph: <https://docs.github.com/en/code-security/concepts/supply-chain-security/dependency-graph>

29. Conclusión ejecutiva

Escaparates Pro posee un activo visual difícil de replicar y una foundation técnica que ya ha superado la fase de “idea”. El error ahora sería seguir añadiendo amplitud sin cerrar identidad, billing, persistencia, delivery y trust.

La estrategia correcta es:

congelar una fuente de verdad auditada;

asegurar sesión, billing y entitlements;

convertir el estado visual en ProjectDocument persistente;

mover assets a storage privado;

compilar releases deterministas y revocables;

medir coste, valor y retención;

incorporar colaboración;

añadir IA sólo cuando el sistema pueda validar, compilar y cobrar sus resultados.

El objetivo de excelencia no es parecer una empresa de Silicon Valley. Es operar con la claridad, los contratos, la evidencia y los gates que permiten a un equipo pequeño —humano y asistido por IA— construir un SaaS fiable sin caer en un prototipo infinito.